

Guard App — Project Overview & Architecture

1. Prerequisites

Requirement	Version
Node.js	18.x or 20.x LTS
npm	9+
Expo CLI	<code>npx expo</code> (SDK 51)
Xcode	15+ (iOS builds)
Android Studio	Hedgehog+ (Android builds)
Physical Device	Required for background location testing

```
npm install -g eas-cli
npx expo install --fix # Fix dependency version mismatches
```

2. Local Setup

1. Install dependencies

```
npm install
```

2. Copy environment template

```
cp .env.example .env.local
```

3. Generate native projects (first time)

```
npx expo prebuild --clean
```

4. Start Metro bundler

```
npx expo start
```

5. Build and run on device (required for location)

```
npx expo run:ios --device
```

```
npx expo run:android --device
```

EAS Build (CI/CD)

```
# Development build
eas build --profile development --platform ios

# Production build
eas build --profile production --platform all
```

3. Environment Variables

Create `.env.local` at the project root. All variables must be prefixed with `EXPO_PUBLIC_` to be accessible in the app bundle.

```
# API
EXPO_PUBLIC_API_URL=https://api.bsecure.in/api/v1
EXPO_PUBLIC_WS_URL=wss://api.bsecure.in/ws

# Maps
EXPO_PUBLIC_GOOGLE_MAPS_API_KEY=AIzaSy...

# Sentry (error tracking)
EXPO_PUBLIC_SENTRY_DSN=https://...@sentry.io/...

# Feature Flags
EXPO_PUBLIC_ENV=production

# Background Location Task Name (constant – do not change per env)
BACKGROUND_LOCATION_TASK=BSECURE_GUARD_LOCATION_TASK
```

Accessing Variables in Code

```
// lib/constants.ts
export const API_URL = process.env.EXPO_PUBLIC_API_URL!;
export const WS_URL = process.env.EXPO_PUBLIC_WS_URL!;
export const GOOGLE_MAPS_KEY =
  process.env.EXPO_PUBLIC_GOOGLE_MAPS_API_KEY!;
export const BACKGROUND_LOCATION_TASK =
  'BSECURE_GUARD_LOCATION_TASK';
```

4. Background Location — Critical Setup

The guard app's defining feature is continuous background location tracking. This requires native configuration in `app.json` and special permissions.

app.json Configuration

```
{
  "expo": {
    "name": "b-secure Guard",
    "slug": "bsecure-guard",
    "version": "1.0.0",
    "orientation": "portrait",
    "icon": "./assets/icon.png",
    "plugins": [
      [
        "expo-location",
        {
          "locationAlwaysAndWhenInUsePermission": "b-secure Guard needs access to your location at all times to share your position with users during active bookings.",
          "locationAlwaysPermission": "b-secure Guard needs your location in the background to track your position during active bookings.",
          "locationWhenInUsePermission": "b-secure Guard needs your location to show you on the map and navigate to users.",
          "isIosBackgroundLocationEnabled": true,
          "isAndroidBackgroundLocationEnabled": true
        }
      ],
      [
        "expo-notifications",
        {
          "icon": "./assets/notification-icon.png",
          "color": "#1B4332",
          "sounds": [".assets/sounds/new_request.wav"]
        }
      ],
      "expo-task-manager"
    ],
    "ios": {
      "bundleIdentifier": "in.bsecure.guard",
      "supportsTablet": false,
      "infoPlist": {
        "NSLocationAlwaysAndWhenInUseUsageDescription": "b-secure Guard needs your location at all times to share your position during active bookings.",
        "NSLocationAlwaysUsageDescription": "b-secure Guard needs your location in the background during active bookings.",
        "NSLocationWhenInUseUsageDescription": "b-secure Guard needs your location to show you on the map.",
        "UIBackgroundModes": ["location", "fetch", "remote-notification"]
      }
    },
    "android": {
      "package": "in.bsecure.guard",
      "permissions": [
```

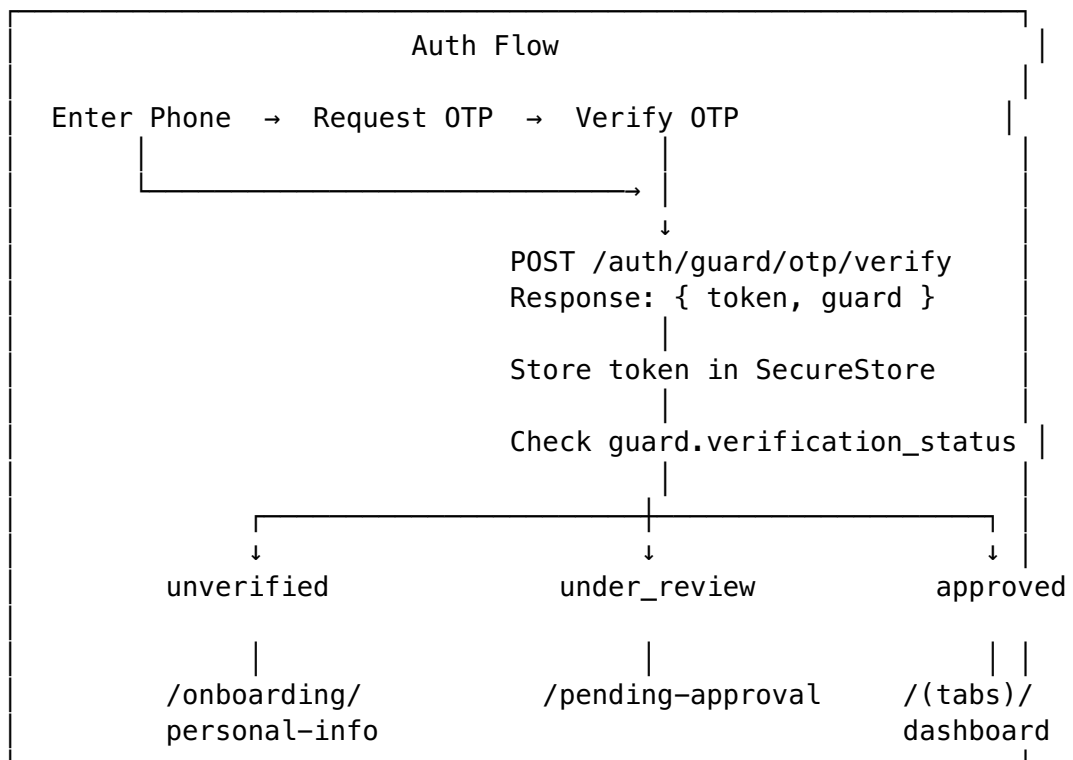
```

    "ACCESS_FINE_LOCATION",
    "ACCESS_COARSE_LOCATION",
    "ACCESS_BACKGROUND_LOCATION",
    "FOREGROUND_SERVICE",
    "FOREGROUND_SERVICE_LOCATION",
    "VIBRATE",
    "RECEIVE_BOOT_COMPLETED"
  ],
  "foregroundService": {
    "notificationTitle": "b-secure Guard is active",
    "notificationBody": "Sharing your location with the user",
    "notificationColor": "#1B4332"
  }
}
}
}
}

```

5. Auth Flow

The guard app uses the same OTP-based authentication as the user app but issues a guard-scoped JWT token.



Root Layout Auth Gate

```

// app/_layout.tsx
import { useEffect } from 'react';

```

```

import { Slot, useRouter, useSegments } from 'expo-router';
import { useGuardStore } from '@stores/guardStore';
import { getStoredToken } from '@lib/storage';

export default function RootLayout() {
  const router = useRouter();
  const segments = useSegments();
  const { guard, setGuard } = useGuardStore();

  useEffect(() => {
    async function checkAuth() {
      const token = await getStoredToken();
      if (!token) {
        router.replace('/(auth)/welcome');
        return;
      }

      // Token exists – fetch guard profile to get current status
      try {
        const profile = await fetchGuardProfile(token);
        setGuard(profile);

        const inAuth = segments[0] === '(auth)';

        if (profile.verification_status === 'unverified') {
          router.replace('/onboarding/personal-info');
        } else if (
          profile.verification_status === 'pending' ||
          profile.verification_status === 'under_review'
        ) {
          router.replace('/pending-approval');
        } else if (profile.verification_status === 'approved') {
          if (inAuth) router.replace('/(tabs)/dashboard');
        }
      } catch {
        router.replace('/(auth)/welcome');
      }
    }

    checkAuth();
  }, []);

  return <Slot />;
}

```

6. Onboarding Gate

After successful OTP verification, the app checks `verification_status` from the guard profile and routes accordingly:

Status	Route	Description
unverified	/onboarding/ personal-info	Guard has never submitted documents
pending	/pending-approval	Documents submitted, not yet reviewed
under_review	/pending-approval	Documents under admin review
approved	/(tabs)/dashboard	Full app access granted
rejected	/onboarding/ documents	Re-upload required with rejection reasons

Onboarding Steps

Step 1: Personal Info	→ POST /onboarding/personal-info/
Step 2: Document Upload	→ POST /documents/upload-url/ (pre-signed)
	→ PUT <s3_url> (direct S3 upload)
	→ POST /documents/confirm/
Step 3: Completion Screen	→ Poll GET /guards/me/verification-status/

7. Zustand Stores

guardStore

```
// stores/guardStore.ts
import { create } from 'zustand';
import { persist, createJSONStorage } from 'zustand/middleware';
import AsyncStorage from '@react-native-async-storage/async-storage';
import type { Guard } from '@types/guard';

interface GuardStore {
  guard: Guard | null;
  isOnline: boolean;
  setGuard: (guard: Guard) => void;
  updateGuard: (partial: Partial<Guard>) => void;
  setOnline: (online: boolean) => void;
  clearGuard: () => void;
}

export const useGuardStore = create<GuardStore>()(
  persist(
    (set) => ({
```

```

    guard: null,
    isOnline: false,
    setGuard: (guard) => set({ guard }),
    updateGuard: (partial) =>
      set((state) => ({
        guard: state.guard ? { ...state.guard, ...partial } :
        null,
      })),
    setOnline: (isOnline) => set({ isOnline }),
    clearGuard: () => set({ guard: null, isOnline: false }),
  }),
  {
    name: 'guard-store',
    storage: createJSONStorage(() => AsyncStorage),
  }
)
);

```

activeBookingStore

```

// stores/activeBookingStore.ts
import { create } from 'zustand';
import type { ActiveBooking, IncomingRequest } from '@types/
  booking';

interface ActiveBookingStore {
  activeBooking: ActiveBooking | null;
  incomingRequest: IncomingRequest | null;
  setActiveBooking: (booking: ActiveBooking | null) => void;
  setIncomingRequest: (request: IncomingRequest | null) => void;
  updateBookingStatus: (status: ActiveBooking['status']) => void;
}

export const useActiveBookingStore =
  create<ActiveBookingStore>((set) => ({
    activeBooking: null,
    incomingRequest: null,
    setActiveBooking: (activeBooking) => set({ activeBooking }),
    setIncomingRequest: (incomingRequest) =>
      set({ incomingRequest }),
    updateBookingStatus: (status) =>
      set((state) => ({
        activeBooking: state.activeBooking
          ? { ...state.activeBooking, status }
          : null,
      })),
  }));

```

earningsStore

```

// stores/earningsStore.ts
import { create } from 'zustand';

```

```

import type { EarningsSummary } from '@types/earnings';

interface EarningsStore {
  summary: EarningsSummary | null;
  setSummary: (summary: EarningsSummary) => void;
  addEarning: (amount: number) => void;
}

export const useEarningsStore = create<EarningsStore>((set) => ({
  summary: null,
  setSummary: (summary) => set({ summary }),
  addEarning: (amount) =>
    set((state) => ({
      summary: state.summary
        ? {
            ...state.summary,
            today: state.summary.today + amount,
            total: state.summary.total + amount,
          }
        : null,
    })),
}));

```

8. React Query Setup

```

// lib/queryClient.ts
import { QueryClient } from '@tanstack/react-query';

export const queryClient = new QueryClient({
  defaultOptions: {
    queries: {
      staleTime: 1000 * 60 * 2, // 2 minutes
      gcTime: 1000 * 60 * 10, // 10 minutes
      retry: (failureCount, error: any) => {
        if (error?.response?.status === 401) return false;
        if (error?.response?.status === 404) return false;
        return failureCount < 2;
      },
      refetchOnWindowFocus: false,
    },
    mutations: {
      retry: false,
    },
  },
});

// app/_layout.tsx (additions)
import { QueryClientProvider } from '@tanstack/react-query';
import { queryClient } from '@lib/queryClient';

```

```
export default function RootLayout() {
  return (
    <QueryClientProvider client={queryClient}>
      {/* ... rest of layout */}
    </QueryClientProvider>
  );
}
```

9. TypeScript Types

// types/guard.ts

```
export type VerificationStatus =
  | 'unverified'
  | 'pending'
  | 'under_review'
  | 'approved'
  | 'rejected';
```

```
export type GuardTier = 'basic' | 'standard' | 'premium' |
  'elite';
```

```
export interface Guard {
  id: string;
  full_name: string;
  phone: string;
  email?: string;
  photo_url?: string;
  date_of_birth?: string;
  address?: string;
  city?: string;
  skills: string[];
  experience_years: number;
  verification_status: VerificationStatus;
  tier: GuardTier;
  rating: number;
  total_bookings: number;
  total_earnings: number;
  is_online: boolean;
  availability_schedule: AvailabilitySchedule;
  created_at: string;
}
```

```
export interface AvailabilitySchedule {
  monday: DaySchedule;
  tuesday: DaySchedule;
  wednesday: DaySchedule;
  thursday: DaySchedule;
  friday: DaySchedule;
  saturday: DaySchedule;
  sunday: DaySchedule;
}
```

```
}
```

```
export interface DaySchedule {  
  enabled: boolean;  
  start_time: string; // "HH:MM"  
  end_time: string;   // "HH:MM"  
}
```